

AI 활용 기술 문서

— Debut Loop!

00 한 눈에 보기

팀 구성	3인 — 개발 1명(박선미), 디자이너 2명(장권익·김지윤). 전원 생성 AI로 산출물 제작
작업 방식	사람이 설계·판단하고, AI가 구현·생성하며, 산출물은 사람이 검증·반려하는 디렉팅 구조
코딩 AI	Claude Code (Anthropic) — Claude Opus
이미지 AI	GPT sol (medium · high) · Gemini Nano Banana → After Effects 편집
영상 AI	Seedance 2.0 · Kling 3.0 → After Effects 편집
음악·효과음	Suno Premier (유료 플랜)
제작 기간	2026-07-17 ~ 08-10

제3자 저작물은 사용하지 않았습니다. 모든 에셋이 생성 AI 산출물이며, 코드는 오픈소스 라이브러리 위에 직접 작성했습니다.

01 AI 활용 구조 (팀 전체)

1-1. 팀 구성과 담당 영역

3인 전원이 각자의 영역에서 생성 AI를 사용했습니다. 도구는 달라도 **작업 방식은 하나의 원칙을 공유**합니다.

담당	영역	사용한 AI	산출물
박선미	기획 · 개발 · 오케스트레이션	Claude Code	게임 코드 전체, 자산 파이프라인, QA 자동화
장권익	캐릭터 · 영상 · 텍스트 연출	GPT sol, Nano Banana, Seedance, Kling	캐릭터 121슬롯, 프롤로그·타이틀·로딩·idle 영상
김지윤	아트 컨셉 · 배경 · GUI	GPT sol, Nano Banana	배경 16슬롯, UI 스킨 141슬롯

1-2. 공통 원칙 — 사람이 정하고, AI가 만들고, 사람이 검증한다

개발과 디자인이 도구는 달라도 **같은 3단계**를 거칩니다. 단계 사이를 잇는 것은 사람의 승인이고, 통과하지 못한 산출물은 ③로 되돌아갑니다.



③에서 나가는 경로는 두 개뿐이고, 둘 다 사람이 고릅니다. 그래서 **AI가 스스로 산출물을 확정하는 지점이 존재하지 않습니다**. 코드는 사람이 승인한 것만 저장소에 들어갔고, 아트는 사람이 선별·보정한 것만 게임에 들어갔습니다.

【1부】개발 — 바이트코딩

02 사람과 AI의 역할 분담

AI에게 "게임을 만들어줘"라고 말하지 않습니다. 사람이 설계 문서와 작업 규약을 먼저 쓰고, AI는 그 범위 안에서 구현하며, 산출물은 사람이 검증·반려하는 구조로 진행했습니다.



왜 이렇게 했나

AI에게 전권을 주면 빠르지만, 무엇이 왜 그렇게 됐는지 사람이 모르게 됩니다. 특히:

- 잘못된 전제 위에 쌓인 구현은 나중에 통째로 버려야 합니다
- 검증 없는 "완료" 보고는 신뢰할 수 없습니다
- 여러 세션이 동시에 작업하면 서로의 변경을 덮어씁니다

그래서 각각 설계 문서 선행, 검증 게이트 강제, 워크트리 격리에 대응하는 규약을 만들었습니다.

03 개발 AI 대상 주요 프롬프트 및 지시 사항

실제로 사용한 지시 문서입니다. 모두 저장소에 그대로 남아 있습니다.

3-1. 작업 규약 (모든 작업에 공통 적용)

- 파일별로 제안 → 승인 → 적용. 한 파일 수정할 때마다 diff를 먼저 보여주고 승인을 받는다. 여러 파일을 한 번에 고치지 않는다.
- 편집 전 반드시 Read. 로컬이 이 문서와 다를 수 있다.
낮선 코드를 되돌리지 말 것 - git diff로 내 변경분과 구분한 뒤 손댄다.
- HARD GATES - 브랜치 생성·머지·배포·파괴적 작업은 명시적 승인 없이 금지.

— docs/QA_HARNESS_PLAN.md §1

커밋 · 푸시 · PR 생성 · PR 머지 - 이 넷은 전부 사용자의 명시적 허락을 받고 실행한다. 승인된 계획 문서에 적혀 있어도 그것을 실행 권한의 근거로 삼지 않는다.

— 세션 간 유지되는 작업 규약

새 브랜치 작업은 main 체크아웃에서 직접 하지 않고, git worktree로 분리한 디렉터리에서 한다. 여러 세션이 동시에 작업하므로 작업 공간을 물리적으로 격리해 오염을 원천 차단한다.

— 동시 작업 중 실제로 다른 세션의 파일이 섞여 들어갈 뻔한 뒤 추가한 규약

3-2. 검증 게이트 (완료로 주장하기 전 반드시 실행)

각 단계 종료 시 반드시:

```
npx tsc --noEmit      # 타입
npx vitest run        # 테스트 110개 유지
```

그리고 게임이 정상 동작하는지 육안 확인 - 하네스가 게임을 망가뜨리지 않았는지가 가장 중요하다.

— docs/QA_HARNESS_PLAN.md §4

말로만 "완료했습니다"라고 하지 못하도록, **명령 출력을 근거로 제시**하게 했습니다.

3-3. 반복 작업의 정형화 — 슬래시 명령

QA처럼 반복되는 작업은 **매번 설명하지 않도록 명령 문서로 고정**했습니다.

/qa-sync (111줄) — 검수 체크리스트를 GitHub 이슈로 등록

1. 시트 읽기 → 행 배열로 정규화
2. 이슈 대상 선별
판정 = Fail → bug
판정 = ? → question
판정 = N-A/Pass + 코멘트 → enhancement
3. 기존 이슈의 마커를 수집해 중복 판별
마커: <!-- qa-row: 10 | fail | a3f9c2 -->
(행번호 | 판정 | 코멘트 해시)
4. 생성 전 대상 목록을 보여주고 승인받는다 ← 반드시
5. gh issue create

/qa-work (132줄) — 이슈를 우선순위로 처리

우선순위:

1. 플레이 영상에 잡히는 화면(프롤로그·로딩·타이틀·로비·스토리)의 bug
2. 나머지 bug
3. enhancement
4. question

라벨별 분기:

- qa:question → 코드를 건드리지 않는다. 조사 후 근거(파일:줄)와 함께 답변만
- qa:asset → 에디터 설정값(JSON)만 수정
- qa:code → 브랜치 생성 → 수정 → 검증 게이트 → PR

공개 범위 규칙:

- 코드 저장소는 public이고 심사 대상이다. 브랜치명·PR 제목·커밋 메시지에
- QA 이슈 번호를 쓰지 않는다. 증상 기반으로 짓는다.
- ❌ qa/issue-12 → ✅ fix/title-hero-crop

라벨로 작업 유형을 나눈 이유는, 그렇게 하지 않으면 **"답변만 필요한 질문"에 AI가 코드를 짜기 때문**입니다.

3-4. 상세 작업 지시서

큰 기능은 착수 전에 설계 문서를 먼저 썼습니다.

문서	분량	내용
docs/QA_HARNESS_PLAN.md	510줄	자산 QA 자동 판정 하네스 — Phase/Task/Step 단위 지시, diff 예시, 롤백 규약
docs/ASSET_EDITOR_PIPELINE.md	334줄	자산 파이프라인 구조 — 슬롯 정의, 업로드 경로, 작업 규약
docs/QA_HARNESS_HANDOFF.md	108줄	다음 에이전트를 위한 인수인계 — 읽는 순서, 실측 현황, 착수 첫 스텝
docs/superpowers/specs/	6개	기능별 설계 문서
docs/qa/README.md	135줄	팀 QA 안내 (디자이너용)

【2부】디자인 — 생성 AI 아트 파이프라인

05 디자인 AI 활용 구조

5-1. 사용 도구와 역할

도구	용도	후처리
GPT sol (medium · high)	초기 컨셉 탐색 · 이미지 생성	After Effects
Gemini Nano Banana	캐릭터 · 배경 · UI · 이벤트 이미지 · 텍스트 디자인	After Effects
Seedance 2.0	프롤로그 · 무대 · 장면 영상	After Effects
Kling 3.0	타이틀 · 로딩 · 캐릭터 idle 루프 영상	After Effects

생성 결과를 그대로 사용하지 않고, **부분 수정·합성·색상 보정·해상도 개선**을 거쳐 최종 에셋으로 완성했습니다.

5-2. 제작 대상 에셋 범위

한 사람이 한 장을 잘 뽑는 작업이 아니라, **서로 다른 용도의 대량 에셋을 한 게임처럼 보이게 만드는 작업**이었습니다.

캐릭터	5인 × (일상복·연습복·무대의상) × (전신·반신) + 표정 세트 + 선택용 원형 초상
배경	연습실 · 기숙사 · 오디션장 · 녹음실 · 공연 무대 · 편의점 · 휴식 공간
UI	포토카드 · 멤버 점검 · 리듬게임 · 춤 연습 · 보컬 녹음 · 카드 짝맞추기 · 사보타주 저지
영상	프롤로그(35초) · 타이틀 · 로딩 · 캐릭터 idle · 무대/리듬게임 소스
텍스트	게임 로고 · 카드 브랜드명 · 프롤로그 자막

5-3. 공통 워크플로

- | | |
|-----------|---|
| ① 목적 정의 | 무엇을 · 어디에 쓸 것인가 (게임 내 사용 위치, 최종 편집 방식) |
| ② 기준 확인 | 캐릭터 기준안 · 프로젝트 톤앤매너와 대조 |
| ③ 프롬프트 작성 | 대상 · 구도 · 감정 · 스타일 · 화면 비율 + 용도별 트리거 워드 |
| ④ 복수 생성 | 하나를 고집하지 않고 여러 방향을 뽑는다 |
| ⑤ 선별 | 목적 적합성 · 캐릭터 일관성 · 실제 적용 가능성 기준으로 비교 |
| ⑥ 반복 개선 | 유지할 요소와 수정할 요소를 나누고, 부족한 부분만 다시 생성 |
| ⑦ 후처리 | 보정 · 합성 · 영상 편집 · 해상도/포맷 최적화 |
| ⑧ 버전 관리 | 원본 · 수정본 · 최종본 구분 보관 |

④~⑥의 선별·반복이 실제 작업 시간의 대부분을 차지했습니다. 생성은 빠르지만, 게임에 넣을 수 있는 수준으로 만드는 것은 선별과 재생성의 반복이었습니다.

06 캐릭터·프로젝트 일관성 관리

생성 AI로 게임 아트를 만들 때 가장 어려운 지점은 **같은 캐릭터가 매번 다르게 나온다**는 것입니다. 이 프로젝트에서 가장 많은 공을 들인 영역입니다.

6-1. 캐릭터 정체성 기준화

캐릭터 5인 각각에 대해 **확정된 대표 이미지**를 먼저 만들고, 이후 모든 생성물을 이 기준과 대조했습니다.

- 캐릭터를 식별할 수 있는 핵심 특징을 기준화 (인상·헤어·실루엣)
- 캐릭터별 시그니처 컬러와 의상 방향 고정
- 반복 제작 시 동일한 캐릭터 설정과 명칭을 그대로 사용
- 캐릭터 차이가 크게 발생한 결과물은 완성도가 높아도 제외

동시에 **5인이 하나의 아이돌 그룹으로 보여야** 하므로, 개별 개성은 유지하되 전체 이미지 스타일과 품질 수준을 통일했습니다.

6-2. 파생 에셋 일관성 — 의상과 표정

기본 표정·기본 의상을 기준 에셋으로 두고, **변경되는 요소와 유지되어야 하는 요소를 분리**했습니다.

축	변경되는 것	유지되는 것
의상	일상복 → 연습복 → 무대의상	캐릭터 기본 디자인·시그니처 컬러
표정	기쁨·불안·슬픔·놀람·결의·각성	캐릭터 고유의 인상
구도	전신 → 반신 → 원형 초상	색감·분위기·품질 수준

표정은 캐릭터 성격에 맞는 감정 범위로 제작해, 스토리 장면에서 바로 쓸 수 있게 했습니다. 개별 에셋 검수 후 **전체 캐릭터 세트를 다시 한 번 통합 검수**하는 2단 검수를 거쳤습니다.

6-3. 영상 내 캐릭터 일관성

영상은 프레임이 진행되면서 인상·의상·신체 표현이 변형될 수 있습니다. 확정된 캐릭터 이미지를 기준으로 결과물을 검수했습니다.

- 움직임 과정에서 발생하는 캐릭터 변형(모핑) 검수
- 프레임별 품질 편차가 두드러지는 결과물 제외
- idle 영상과 무대 영상처럼 성격이 다른 콘텐츠도 기존 이미지와 연결되는지 확인

6-4. 프로젝트 전체 톤앤매너

배경 7종과 UI 7종은 각각 다른 장소·다른 기능이지만, **하나의 모바일 아이돌 게임 화면**으로 보여야 합니다.

- 프로젝트 공통 톤앤매너와 대표 색상 설정
- 캐릭터 · 배경 · UI 사이의 색감과 분위기 상호 검토
- 동일한 화면 비율과 해상도 기준 적용
- 결과물마다 표현 수준(선명도·마감)이 지나치게 달라지지 않도록 조정

6-5. 검수·제외 기준

위 네 축에 공통으로 적용한 판단 기준입니다.

- 제외: 기존 확정 에셋과 스타일 차이가 큰 결과물
캐릭터 식별이 어려울 만큼 변형된 결과물
과도한 장식이 추가되어 화면 위계를 해치는 결과물
프레임별 품질 편차가 큰 영상
- 채택: 기존 에셋과의 연결성이 확보되고
캐릭터·텍스트·UI가 결합된 상태에서도 활용 가능한 결과물

07 프롬프트 설계와 트리거 워드

7-1. 프롬프트 구성 요소

"원하는 이미지를 설명한다"가 아니라, **요소를 분해해서 지정**하는 방식으로 작성했습니다.

- | | |
|--------|-----------------------------|
| 대상 | 무엇을 제작하는가 (인물의 역할·성격, 오브젝트) |
| 장면 목적 | 이 이미지가 게임에서 무엇을 전달해야 하는가 |
| 감정·분위기 | 장면이 전달할 정서 |
| 구도 | 피사체의 배치와 카메라 방향 |
| 스타일 | 게임 전체 비주얼 기준 |
| 화면 비율 | 최종 적용 환경 (모바일 세로 9:16) |
| 제외 요소 | 들어가면 안 되는 것 |

원칙:

- **중요한 조건을 앞세우고**, 부차적 요소는 줄인다
- 서로 충돌하거나 해석이 모호한 표현은 제거한다
- 필요한 요소와 **제외해야 할 요소를 함께** 명시한다

7-2. 용도별 프롬프트 분기

같은 도구를 쓰더라도 **게임 내 사용 위치와 최종 편집 방식에 따라 프롬프트 방향을 다르게** 설정했습니다.

캐릭터	인물의 역할·성격, 의상, 감정, 장면 내 위치
배경	캐릭터와 UI가 함께 배치될 것을 전제로 한 여백과 시선 흐름
UI	기능성과 편집 가능성 — 실제 정보가 들어갈 공간 확보
영상	외형 묘사가 아니라 장면 안에서 일어나는 움직임과 변화
텍스트	문구의 정확성과 화면 내 위계

특히 UI는 완성된 화면뿐 아니라 **캐릭터·텍스트·아이템을 나중에 배치할 수 있는 빈 플레이 영역(클린 이미지)**을 별도로 제작했습니다.

7-3. 트리거 워드 체계

핵심 조건이 생성 과정에서 흐려지지 않도록 용도별 트리거 워드를 정리해두고, **모든 프롬프트에 넣는 고정 문장이 아니라 목적에 따라 선택적으로** 사용했습니다.

이미지		영상	
캐릭터 유지	preserve character identity	캐릭터 유지	preserve character identity
기존 디자인 유지	keep the existing appearance and outfit	자연스러운 동작	subtle natural motion
세로형 게임 화면	vertical 9:16 mobile game composition	안정적인 카메라	stable camera
게임 일러스트 스타일	high-quality modern anime game visual	화면 구성 유지	maintain the original composition
편집 가능한 UI	clean UI, blank usable area	반복 영상	seamless loop
		텍스트	
		정확한 문구	exact text, verbatim
		가독성	clearly readable
		배치	centered alignment, safe margin

7-4. 네거티브 제약

생성 범위가 넘치지 않도록 **제외 조건도 프롬프트의 일부**로 관리했습니다.

이미지	no extra character, no logo, no watermark
영상	no character morphing, no outfit change
텍스트	no other text, no fake writing

특히 텍스트는 AI가 임의의 글자를 만들어내는 문제가 잦아, 네거티브 제약의 효과가 컸습니다.

08 영상 제작

8-1. 영상 프롬프트 원칙 — 외형이 아니라 움직임

정지 이미지의 외형 설명을 반복하는 대신, **장면 안에서 어떤 움직임과 변화가 필요한지**를 전달하는 데 집중했습니다.

- 영상의 목적과 게임 내 활용 위치를 먼저 정의
- 목적에 맞는 움직임의 **속도와 강도** 조절
- 한 장면에 과도한 사건이나 움직임이 들어가지 않도록 구성
- 반복 영상은 연결성을 고려해 제작

8-2. 카메라·동작 설계

캐릭터 동작과 카메라 움직임이 **동시에 과도해지면 화면이 불안정**해집니다. 장면 목적에 따라 한쪽을 눌렀습니다.

장면 성격	카메라	캐릭터 동작
표정·감정 전달	안정 유지	미세한 자연스러움
무대·사건 연출	흐름 부여	역동적

역동적인 장면에서도 **주요 피사체의 가시성**을 우선했습니다.

8-3. 프롤로그 시네마틱 (35초)

게임 도입부와 핵심 사건을 전달하는 35초 영상입니다. 하나의 긴 영상을 한 번에 생성하지 않고, **장면의 역할과 감정 흐름에 맞는 소스를 개별 생성한 뒤 편집으로 연결**했습니다.

- 스토리 흐름에 따라 장면 단위로 소스 제작
- 각 영상은 개별 장면으로도 완성도를 갖추되, 연결 시 캐릭터·배경·색감이 분리되지 않도록 검수
- 서로 다른 생성 영상 사이의 색감과 분위기 통일 (After Effects 보정)
- 내레이션과 자막이 들어갈 화면 영역을 미리 비워둠
- 복수 시안을 비교해 최종 소스 선정

8-4. 루프·게임 적용 영상

타이틀 · 로딩 · 캐릭터 idle 등 **반복 재생되는 영상**은 화려한 전환보다 **자연스러운 반복성과 안정성**을 우선했습니다.

- 시작과 종료가 자연스럽게 이어지도록 구성 (**seamless loop**)
- 반복 과정에서 화면 변화가 커지지 않도록 관리
- 영상 위에 로고 · 버튼 · 로딩 표시 · 대사가 없이는 것을 전제로 화면 구성
- **편집용 원본과 게임 적용용 결과물**을 분리해 관리

09 텍스트·로고 디자인

9-1. 문자 정확성 제어

로고·카드 브랜드명·자막처럼 글자가 들어가는 콘텐츠는, 시가 **임의의 글자를 추가하거나 문구를 변형**하는 문제가 있습니다. 표시할 문구를 명확히 지정하고 네거티브 제약을 함께 걸었습니다.

- 아이돌 카드의 ZERO:C — 메인 심볼을 방해하지 않는 **브랜드 서명 형태**로 적용
- 메인 화면의 Debut Loop! 로고 — 캐릭터 및 게임 시작 영역과의 균형을 고려해 배치

텍스트가 캐릭터나 메인 심볼보다 강조되지 않도록 **화면 안에서의 역할과 중요도**를 함께 지정했습니다.

9-2. 자막 위계 설계

프롤로그 내레이션을 전부 같은 자막으로 처리하지 않고, **문장의 기능과 감정적 중요도에 따라 디자인을 구분**했습니다.

문장 유형	예시	시각적 강도
일반 상황 설명	—	기본
사건 강조	"붉은 섬광", "꺼진 마이크"	강
시간·회귀 표현	"3년 전", "0시"	강
주인공 내면 독백	"사고가 아니었다"	강

세로형 영상에서 **주요 피사체를 가리지 않는 위치**를 선정하고, 장면과 조화를 이루는 색상을 적용했습니다.

9-3. 편집용 투명 오버레이 분리

텍스트를 영상에 굽지 않고, **투명 배경 오버레이 소스를 별도 제작**했습니다.

- 편집 단계에서 위치·크기·등장 타이밍을 자유롭게 조정
- 자막 포함 영상과 독립 오버레이 소스를 분리 보관
- 후반 작업을 고려한 고해상도 텍스트 소스

10 게임 적용 최적화와 자산 관리

생성한 에셋이 실제로 **게임 안에서 동작하는 것까지**가 디자인 작업 범위였습니다.

10-1. 화면 규격

- 모바일 세로 **9:16** 기준으로 생성 단계부터 비율 지정
- UI가 얹힐 것을 전제로 **안전영역과 여백** 확보
- 화면 내 캐릭터 크기와 배치 방향 관리
- 최종 사용 환경에서 가독성과 배치 상태를 실기로 확인

10-3. 버전 관리

원본·수정본·최종본을 구분해 보관하고, 파생 에셋(표정·의상·구도별)도 세트 단위로 관리했습니다. 실제 게임 반응은 **슬롯 기반 자산 에디터**를 통해 디자이너가 직접 업로드하는 구조라, 코드 수정 없이 아트를 교체할 수 있었습니다.

10-2. 플랫폼별 포맷

알파 채널 영상은 브라우저별 코덱 지원이 달라 **두 벌로 제작**했습니다.

Chrome·웹	WebM (VP9 alpha)
iOS·Safari	HEVC MOV (alpha)
정지 이미지	WebP (배포 최적화)

11 에셋 · 오픈소스 출처

11-1. 외부 에셋

제3자 저작물 없음. 전부 생성 AI 산출물입니다.

구분	생성 도구	후처리	수량
UI 스킨	GPT sol · Gemini Nano Banana	After Effects	141 슬롯
배경	GPT sol · Gemini Nano Banana	After Effects	16 슬롯
캐릭터	GPT sol · Gemini Nano Banana	After Effects	121 슬롯
영상 (타이틀·프롤로그·로딩·idle 루프)	Seedance 2.0 · Kling 3.0	After Effects	위 슬롯에 포함
배경음악 · 효과음	Suno Premier (유료 플랜)	—	11 트랙

- **이미지·영상**: 생성 후 After Effects에서 합성·모션·루프 처리. 알파 영상은 Chrome용 webm과 iOS용 HEVC mov 두 벌로 제작
- **음악**: Suno Premier 유료 구독 플랜으로 생성. 해당 플랜의 상업적 이용 조건을 따름
- **폰트**: 시스템 기본 폰트만 사용 (별도 웹폰트 없음)

파일 단위 목록은 src/data/uiskins.json, backgrounds.json, charskins.json, bgm.json 매니페스트에 슬롯별로 기록되어 있습니다.

11-2. 오픈소스

패키지	버전	용도	라이선스
pixi.js	^8.0.0	2D 렌더링 엔진	MIT
vite	^5.2.0	빌드 도구·개발 서버	MIT
typescript	^5.4.0	정적 타입	Apache-2.0
vitest	^1.5.0	테스트 러너	MIT
@capacitor/core·android·ios	^8.4.2	모바일 패키징	MIT
playwright-core	^1.61.1	화면 검증 자동화	Apache-2.0

빌드 도구로 사용한 외부 실행 파일:

도구	용도	라이선스
ffmpeg	알파 영상 변환 (webm·HEVC mov)	LGPL/GPL
cwebp (libwebp)	이미지 WebP 변환	BSD
sips (macOS 기본)	배경 이미지 규격화	—

12 AI 활용의 한계와 사람의 개입

정직하게 적습니다. AI가 전부 해낸 것은 아닙니다.

12-1. AI가 잘한 것

개발	측정과 계산 — 뷰포트별 잘림량, 색상 실측, 파일 규모 집계
	반복 작업 — 슬롯 배선, 매니페스트 갱신, 캡처 비교
	회귀 검증 — 매 수정마다 타입체크·테스트·화면 확인
디자인	대량 생성 — 하나의 컨셉에서 여러 방향의 시안을 빠르게 확보
	파생 확장 — 확정된 캐릭터에서 의상·표정·구도 변형
	장면 소스 — 편집으로 연결할 영상 조각의 개별 생성

12-2. 사람이 개입해야 했던 것

- 방향 결정 — 아트를 고칠지 뷰어를 고칠지 같은 판단
- 사양 확정 — "카드가 보이는 건 의도된 연출"처럼 기획 의도가 걸린 결정
- 미적 판단 — 폰트 크기, 색상 톤, 배치가 보기 좋은지
- 기준 수립 — 캐릭터 기준안과 톤앤매너는 사람이 먼저 정해야 AI가 따라올 수 있음
- 선별 — 생성 결과 중 무엇이 "이 게임의 것"인지는 사람만 판단 가능
- 잘못된 자신감 교정 — AI가 초기에 "세로 연장은 효과 없다"고 단정했으나, 실기에서는 유효했던 부분을 사람이 지적해 정정

12-3. 도메인 지식이 필요한 이유

AI가 생성을 대신해준 만큼, **무엇을 요구할지 아는 능력**의 비중이 커졌습니다.

- 세로 9:16 · 안전영역 · 알파 채널 코덱 차이를 모르면, 예뻐지만 게임에 넣을 수 없는 에셋이 나옵니다
- 캐릭터 시트 · 표정 세트 · 클린 UI 같은 **제작 관행을 알아야** 프롬프트를 그 단위로 쪼갤 수 있습니다
- 슬롯 · 매니페스트 · 회귀 테스트 같은 개발 개념을 모르면, AI가 낸 "완료"를 검증할 방법이 없습니다

AI는 실행을 압축했지만 기준을 대신 세워주지는 않았습니다. 어디서 사람이 개입해야 하는지를 알고 설계하는 것 — 그것이 이 프로젝트에서 확인한 생성 AI 실무의 핵심이었습니다.